Project Title: FloraGuard: Intelligent Environmental & Soil Health Monitor
Team Name: Rubus Labs
Theme: IoT is Bold

---

1. Project Overview

1.1 Project Motivation and Description

Consistent monitoring of indoor plant health is challenging in modern lifestyles, as invisible environmental fluctuations often cause irreversible strain before physical symptoms manifest.

This project is motivated by the need to bridge the gap between human neglect and plant health. It utilises the **Raspberry Pi 4 Model B** platform integrated with the **Raspberry Pi Sense HAT Version 2** add-on board to create a localised environmental monitoring system.

The system functions by utilising the Sense HAT's integrated sensor suite the **HTS221** for humidity and temperature and the **TCS3400 RGB** (or compatible TCS-series) **color sensor** for measuring ambient luminosity. This real-time data is processed locally by a Python script that evaluates the inputs against safety thresholds. If an anomaly is detected, the system provides immediate feedback via an **8x8 LED matrix**, alerts the user via **automated email**, and maintains a continuous record in a **CSV file**.

1.2 Target Users

This system is designed for three primary groups:

- **Indoor Gardening Enthusiasts:** Individuals who manage flora in their homes that require strict adherence to specific temperature and humidity ranges.
- **Urban Apartment Dwellers:** People with limited natural light or inconsistent climate control who need automated alerts to manage their plant's micro-environment.
- **Commercial Greenhouse Gardeners:** Small-scale food growers who require a low-cost solution for logging environmental data and monitoring plant health.

2: Functional Requirements

2.1 Environmental Data Acquisition

How the system gathers information from the environment.

- **FR01 :** The system shall sample temperature, humidity, and ambient light levels from the Sense HAT sensor every 10 seconds.
- **FR02:** The system shall apply a three-sample moving average to all incoming sensor data to filter out temporary fluctuations and noise.

    - The system maintains a FIFO (First-In-First-Out) buffer of the last three readings to calculate a continuous arithmetic mean before evaluating thresholds.
- **FR03:** The system shall log timestamped sensor data (temperature, humidity, and light intensity) to a local CSV file at every sampling interval (10 seconds).
    - Data is appended to the file in every loop iteration to ensure a continuous record.
    - The log file shall be named `floraguard_log.csv`.
    - Each entry must include Timestamp, Temperature (°C), Humidity (%), Light (Lux), and System Status.

2.2 Threshold Analysis & Logic

- **FR04:** The system shall trigger an “Anomaly State” whenever the `problems` list is non-empty (at least one sensor reading is outside the defined thresholds).
    - **Temperature**: Low threshold **15°C**, High threshold **28°C**.
    - **Humidity**: Low threshold **40%**, High threshold **70%**.
    - **Light Intensity**: Minimum required **50 Lux**.

2.3 User Feedback & Output

- **FR06:** The system shall display a green check mark on the Sense HAT LED 8x8 matrix during normal operation (readings within thresholds).
- **FR07:** When an "Anomaly State" is active, the system shall display a red scrolling text message on the LED matrix identifying the specific threshold breach.
    - **Temperature**: "TOO HOT" or "TOO COLD"
    - **Humidity**: “TOO HUMID” or “TOO DRY”
    - **Light Intensity**: “LOW LIGHT”
- **FR08:** The system shall dispatch a one-time automated email alert upon detecting an "Anomaly State" to notify the user of specific threshold breaches.
    - An email is only sent during the initial transition from a "Normal" to an "Anomaly" state.
    - To prevent inbox flooding, no further emails are sent as long as the system remains in "Anomaly State."
    - The email alert system automatically resets only after all environmental parameters have returned to "Normal" levels.

Section 3: Non-Functional Requirements

Reliability and Availability

- **NFR01:** The system shall maintain a continuous uptime of 99% during the hackathon showcase event.
- **NFR02:** In the event of a momentary power loss, the processing script shall automatically initialize upon Raspberry Pi reboot to resume monitoring without user intervention.

Performance and Response Time

- **NFR03:** The system shall update the LED matrix feedback within 0.5 seconds of a sensor reading being processed.
- **NFR04:** An alert email shall be sent to the user within 3 seconds of a threshold breach event, with delivery to the user's inbox occurring within 10 seconds.

Data Integrity and Accuracy

- **NFR05:** Temperature readings shall be accurate to within 0.5°C within the range of 15°C to 40°C, as specified by the STMicro HTS221 sensor documentation.
- **NFR06:** Humidity readings shall be accurate to within 3.5%rH within the range of 20% to 80% rH as specified by the STMicro HTS221 sensor documentation.
- **NFR07:** To ensure data integrity, the system shall verify that the log.csv file is not corrupted before each write operation.

Storage and Capacity

- **NFR08:** The system shall be capable of storing at least 10,000 individual sensor logs (approx. 2.5MB) on the local SD card before requiring manual data export.

Hardware and Environmental Constraints

- **NFR09:** The system must be powered by a standard 5V/3A USB-C power supply to ensure CPU stability during peak processing.

Section 4: Technical Architecture

4.1 System Architecture Overview

The system follows a modular architecture designed for real-time sensing and responsive action. The architecture is divided into three primary layers: Data Acquisition (Inputs), Central Processing, and Output.

4.2 Hardware Components

The hardware stack is selected to meet the requirements for environmental sensing:

- **Central Processor: Raspberry Pi 4 Model B** (Quad-core ARM Cortex-A72) serves as the primary compute unit for running the Python logic and managing I/O operations.
- **Sensor Interface:** The Sense HAT provides a comprehensive environmental suite, including the **HTS221** (humidity/temperature), **LPS25H** (pressure), and **TCS3400** (color/light) sensors. These components communicate with the Raspberry Pi 4 via the **I2C protocol**.
- **Visual Output:** The **8x8 LED Matrix** on the Sense HAT provides real-time status indicators (Green/Red) to the user.
- **User Alert:** The system sends an alert message to the user via email during threshold breaches.

4.3 Software Logic & Data Flow

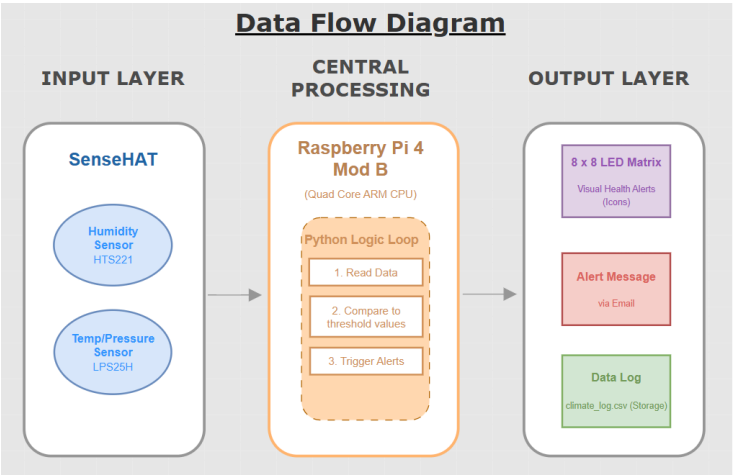
The system operates on a continuous execution loop written in **Python**, utilizing the official `sense-hat` library.

1. **Input Phase:** The script queries the Sense HAT sensors at a fixed 10-second interval.
2. **Processing Phase:** Raw data is passed through a moving-average filter to handle false positives (sensor noise) and then compared against the preconfigured threshold values. These values for temperature, humidity, and light are stored as constants within the Python script's configuration section.
3. **Output Phase:**
    - **Normal State:** LED matrix displays a steady green indicator; data is appended to `floraguard_log.csv`.
    - **Anomaly State:** LED matrix scrolls a red alert message; an alert email is sent to the user (only when the status changes to prevent spamming); the event is flagged in the CSV log.

4.4 Data Persistence Structure

To satisfy the feedback regarding storage planning, the system uses a flat-file structure:

- **Naming Convention:** Logs are saved as `floraguard_log.csv` to ensure simple local data persistence.



Section 5: Success Criteria

5.1 Technical Functional Success

- **Sensor Accuracy:** The system shall record temperature readings that deviate no more than 0.5°C from a calibrated reference within the range of 15°C to 40°C, as specified by the HTS221 sensor documentation.
- **Humidity Precision:** The system shall record relative humidity levels with a maximum deviation of 3.5% rH.

- **Logic Validation:** In 100% of test cases where the defined threshold is breached, the system must update the LED matrix status and initiate an alert email within **10 seconds** (matching the sampling interval).
- **Filter Efficiency:** The moving-average algorithm shall successfully ignore single-point sensor "spikes" (fluctuations lasting < 10 seconds) to prevent false-positive alerts.

5.2 Data Management Success

- **Log Persistence:** The system shall successfully generate a `floraguard_log.csv` file containing at least 1080 entries (3 hours of 10-second interval data) without data corruption or skipped intervals.
- **Storage Stability:** The system shall verify the existence of the log file before each write operation to ensure no data loss occurs during monitoring.

5.3 User Experience & Interaction Success

- **Visual Latency:** The Sense HAT LED Matrix must transition from the “Optimal” checkmark to red error message one processing cycle (10 seconds) of an anomaly detection.
- **Alert Reliability:** The system shall suppress duplicate email alerts, sending only one notification per status change to ensure a positive user experience.

Section 6: Risk Analysis

Risk Category	Potential Issue	Mitigation Strategy
Hardware	Inaccurate sensor readings due to CPU self-heating from the Raspberry Pi 4.	Implement a calibration offset in Python or use physical stand-offs to isolate the Sense HAT from the CPU.
Connectivity	Email alert delivery failure due to unstable or congested Wi-Fi networks.	Maintain the LED matrix as the primary alert mechanism; utilize a mobile hotspot for critical email alerts.
Environmental	Moisture from plant irrigation causing a short circuit on the GPIO pins.	Secure all hardware components in a non-conductive enclosure.
Power failure	Monitoring interruption during transport or unexpected power loss.	Ensure system compatibility with a portable power bank (5V/3A output) for redundant power.
Data Integrity	SD Card corruption or file system failure due to frequent write cycles or improper shutdowns.	Utilize `os.path` verification and "append" mode in Python to ensure log integrity, supported by the power bank to prevent abrupt power loss.

References and Resources

Hardware Documentation

- **Raspberry Pi 4 Model B:** Raspberry Pi Ltd. (2024). *Raspberry Pi 4 Model B Product Brief*. Available at: https://www.raspberrypi.com/documentation/.
- **Raspberry Pi Sense HAT:** Raspberry Pi Ltd. (2023). *Raspberry Pi Sense HAT Product Brief (RP-008384-DS-1)*. Available at: https://pip-assets.raspberrypi.com/categories/676-raspberry-pi-sense-hat/documents/RP-008384-DS-1-sense-hat-product-brief.pdf.
- **Raspberry Pi Sense HAT:** Raspberry Pi Ltd. (2024). *Sense HAT Technical Specifications*. Available at: https://sense-hat.readthedocs.io/en/latest/.

Software Libraries and Frameworks

- **Sense HAT Python API:** *Sense HAT API Reference*. Available at: https://projects.raspberrypi.org/en/projects/getting-started-with-the-sense-hat.
- **Python CSV Module:** Python Software Foundation. *CSV File Reading and Writing*. Available at: https://docs.python.org/3/library/csv.html.

Standards and Guidelines

- **University of Galway:** School of Computer Science. (2025). *Comprehensive Hackathon Guide: "PI the IoT" & "Old is Gold" Version 1.0.*.
- **Jama Software:** *Requirements Management Guide: Writing Functional Requirements*. Available at: https://www.jamasoftware.com/.
- **EARS (Easy Approach to Requirements Syntax):** Mavin, A. (2009). *The EARS Method for Requirements Engineering.*.

Phase-2 Functional Requirement Document Rubric:

Criterion	Excellent (5)	Good (4)	Satisfactory (3)	Needs Work (2)	Poor (1)
Requirement Clarity	All requirements are specific, measurable, and testable	Most requirements clear with minor ambiguity	Requirements present but some lack specificity	Many requirements unclear or vague	Requirements poorly defined or missing
Completeness	All functional and non-functional requirements documented	Most categories covered with minor gaps	Some requirements documented; gaps exist	Significant gaps in documentation	Incomplete or minimal documentation
Technical Feasibility	Project scope is well-matched to timeline and team capability	Generally feasible with minor concerns	Mostly feasible with some challenges	Some feasibility concerns not addressed	Scope unclear or unrealistic
Architecture Design	Clear, well-reasoned system design with appropriate technology choices	Logical design with mostly appropriate technology	Basic design present with some tech choices unexplained	Design lacks clarity; tech choices not justified	No clear architecture provided